

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2**



Stack dan Queue

Oleh:

Nashry Akbar

NIM.2510817210014

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 2

Laporan Praktikum Algoritma & Struktur Data Modul 2: Stack dan Queue ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nashry Akbar
NIM : 2510817210014

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom.,
M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL.....	5
SOAL 1.....	6
A. Pembahasan.....	6
SOAL 2.....	7
A. Source Code.....	7
B. Output Program.....	10
C. Pembahasan.....	13
SOAL 3.....	17
A. Source Code.....	17
B. Output Program.....	20
C. Pembahasan.....	22
LINK GITHUB.....	27

DAFTAR GAMBAR

Gambar 2.1 Output Program Ketika Memilih Push.....	10
Gambar 2.2 Output Program Ketika Memilih Pop.....	11
Gambar 2.3 Output Program Ketika Memilih Cetak Stack.....	11
Gambar 2.4 Output Program Ketika Memilih Bersihkan Stack.....	12
Gambar 2.5 Output Program Ketika Memilih Quit.....	12
Gambar 3.1 Output Program Ketika Memilih Insert.....	20
Gambar 3.2 Output Program Ketika Memilih Delete.....	21
Gambar 3.3 Output Program Ketika Memilih Cetak Queue.....	21
Gambar 3.4 Output Program Ketika Memilih Reset Queue.....	22
Gambar 3.5 Output Program Ketika Memilih Quit.....	22

DAFTAR TABEL

Tabel 2. Source Code Program Soal Nomor 1.....	7
Tabel 3. Source Code Soal Nomor 3.....	17

SOAL 1

Apa perbedaan Stack dengan Queue?

A. Pembahasan

Dalam algoritma dan struktur data, stack dan queue adalah dua jenis struktur data dasar yang mengatur cara data disimpan dan diambil. Perbedaan utama antara Stack dan Queue terletak di cara keluar-masuknya data.

Stack (tumpukan) adalah struktur data yang bekerja dengan prinsip LIFO (*Last In, First Out*), artinya data yang terakhir dimasukkan, akan menjadi data yang pertama saat dikeluarkan. Operasi utama pada stack, meliputi *push* (menambahkan data dari atas), dan *pop* (mengambil data dari tumpukan paling atas). Stack dapat dianalogikan seperti tumpukan piring . Setiap piring yang baru dicuci akan diletakkan di bagian paling atas (*push*). Ketika akan digunakan, piring diambil dari bagian paling atas (*pop*). Dengan demikian, piring yang terakhir diletakkan akan menjadi piring pertama yang diambil, sesuai dengan prinsip LIFO.

Sedangkan, Queue (antrean) adalah struktur data yang bekerja dengan prinsip FIFO (*First In, First Out*), artinya data yang pertama masuk merupakan data yang pertama keluar. Operasi utama pada queue, meliputi *enqueue* (menambahkan data dari belakang antrean), dan *dequeue* (mengambil data dari sisi terdepan antrean). Queue dapat dianalogikan seperti antrean mahasiswa di laboratorium saat akan menggunakan alat. Mahasiswa yang datang lebih awal akan berada di posisi depan antrean dan dilayani terlebih dahulu (*dequeue*), sedangkan mahasiswa yang datang berikutnya akan masuk ke bagian belakang antrean (*enqueue*). Dengan demikian, urutan pelayanan mengikuti urutan kedatangan, sesuai dengan prinsip FIFO.

SOAL 2

Cobalah contoh program berikut, running dan analisis hasilnya!

A. Source Code

Tabel 2. Source Code Program Soal Nomor 1

1	#include <iostream>
2	#include <conio.h>
3	#include <stdlib.h>
4	
5	#define MAX 20
6	
7	using namespace std;
8	
9	struct Stack
10	{
11	int Atas;
12	int Data[MAX];
13	};
14	
15	Stack Tumpuk;
16	
17	int Kosong()
18	{
19	if (Tumpuk.Atas == -1)
20	{
21	return 1;
22	}
23	else
24	{
25	return 0;
26	}
27	}
28	
29	int Penuh()
30	{
31	if (Tumpuk.Atas == MAX-1)
32	{
33	return 1;
34	}
35	else
36	{
37	return 0;
38	}
39	}

```

40
41 void Push(int Data)
42 {
43     if (Kosong()==1)
44     {
45         Tumpuk.Atas++;
46         Tumpuk.Data[Tumpuk.Atas] = Data;
47         cout << "Data " <<
Tumpuk.Data[Tumpuk.Atas] << " dimasukkan ke
dalam Stack." << endl;
48     }
49     else if (Penuh()==0)
50     {
51         Tumpuk.Atas++;
52         Tumpuk.Data[Tumpuk.Atas] = Data;
53         cout << "Data " <<
Tumpuk.Data[Tumpuk.Atas] << " dimasukkan ke
dalam Stack." << endl;
54     }
55     else
56     {
57         cout << "Stack telah Penuh!!!" << endl;
58     }
59 }
60
61 void Pop()
62 {
63     if (Kosong()==0)
64     {
65         cout << "Data " <<
Tumpuk.Data[Tumpuk.Atas] << " diambil dari
Stack."<< endl;
66         Tumpuk.Atas--;
67     }
68     else
69     {
70         cout << "Stack Kosong!!!" << endl;
71     }
72 }
73
74 void Cetak_Stack()
75 {
76     if (Kosong()==0)
77     {
78         for(int i = Tumpuk.Atas; i >= 0; i--)

```

```

79         {
80             cout << "\nTumpukan Ke " << i << " =
            " << Tumpuk.Data[i];
81         }
82         cout << endl;
83     }
84     else
85     {
86         cout << "Stack Kosong!!!" << endl;
87     }
88 }
89
90 void Bersihkan_Stack()
91 {
92     Tumpuk.Atas = -1;
93     cout << "Stack telah dibersihkan!!!" << endl;
94 }
95
96 void Inisialisasi()
97 {
98     Tumpuk.Atas = -1;
99 }
100
101 int main()
102 {
103     Inisialisasi();
104     int Pilihan, Data;
105     do{
106         cout << "\nSTACK" << endl;
107         cout << "=====" << endl;
108         cout << "1. PUSH" << endl;
109         cout << "2. POP" << endl;
110         cout << "3. CETAK STACK" << endl;
111         cout << "4. BERSIHKAN STACK" << endl;
112         cout << "5. QUIT" << endl;
113         cout << "PILIHAN : "; cin >> Pilihan;
114         switch (Pilihan)
115         {
116             case 1:
117                 cout << "Masukkan Nilai: "; cin >>
Data;
118                 Push(Data);
119                 break;
120
121             case 2:

```

```

122         Pop();
123         break;
124
125     case 3:
126         Cetak_Stack();
127         break;
128
129     case 4:
130         Bersihkan_Stack();
131         break;
132
133     default:
134         cout << "TERIMA KASIH" << endl;
135         break;
136     }
137     cout << "Press any key to continue";
138     getch();
139     system("cls");
140 }
141 while (Pilihan < 5);
142 return 0;
143 }

```

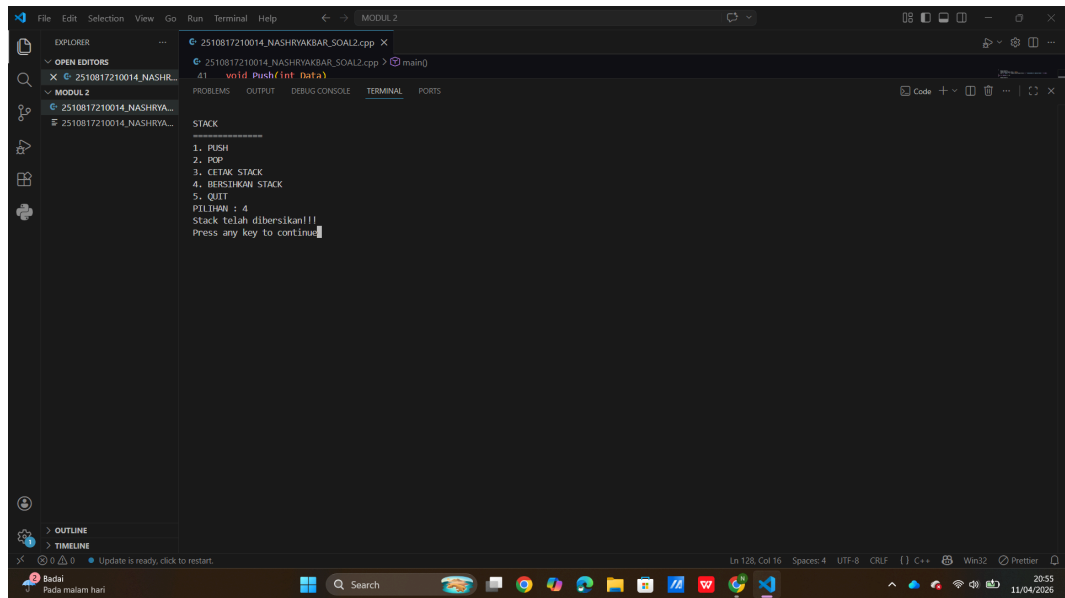
B. Output Program

```

PS C:\Users\USER\OneDrive\Documents\Github\Praktikum-Algoritma\Modul_2> cd "C:\Users\USER\OneDrive\Documents\Github\Praktikum-Algoritma\Modul_2" & if ($?) { g++ 2510817210014_NASHRYAKBAR_SOAL2.cpp -o 2510817210014_NASHRYAKBAR_SOAL2.exe ; if ($?) { .\2510817210014_NASHRYAKBAR_SOAL2.exe }
STACK
=====
1. PUSH
2. POP
3. CETAK STACK
4. BERSIHKAN STACK
5. QUIT
PILIHAN : 1
Masukkan Nilai: 20
Data 20 dimasukkan ke dalam Stack.
Press any key to continue.

```

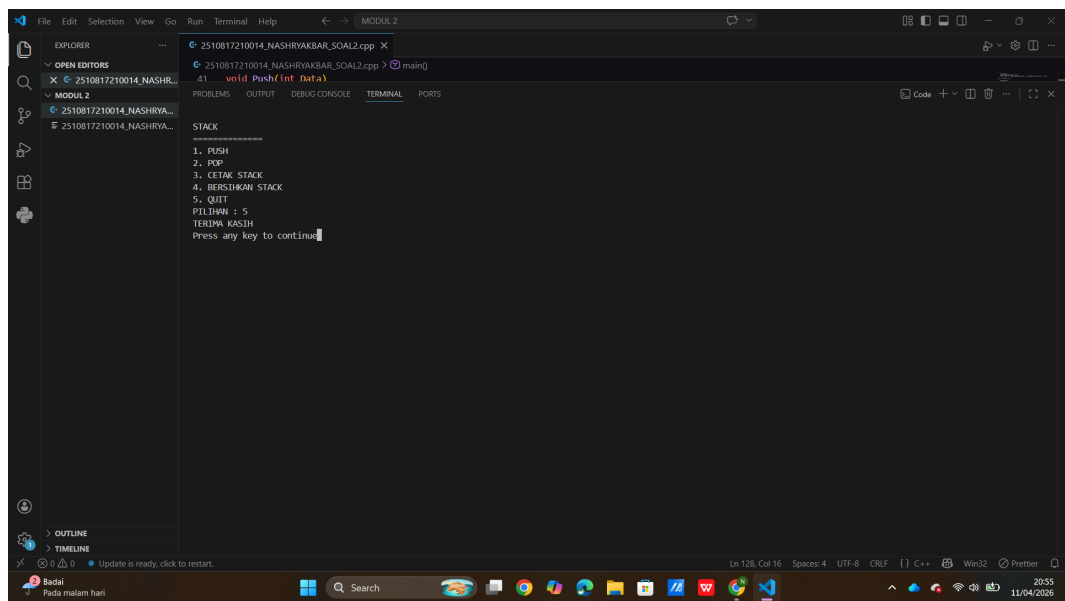
Gambar 2.1 Output Program Ketika Memilih Push



```
2510817210014_NASHRYAKBAR_SOAL2.cpp
2510817210014_NASHRYAKBAR_SOAL2.cpp > main()
41 void Push(int data)

STACK
-----
1. PUSH
2. POP
3. CETAK STACK
4. BERSIHKAN STACK
5. QUIT
PILIHAN : 4
Stack telah dibersihkan!!!
Press any key to continue
```

Gambar 2.4 Output Program Ketika Memilih Bersihkan Stack



```
2510817210014_NASHRYAKBAR_SOAL2.cpp
2510817210014_NASHRYAKBAR_SOAL2.cpp > main()
41 void Push(int data)

STACK
-----
1. PUSH
2. POP
3. CETAK STACK
4. BERSIHKAN STACK
5. QUIT
PILIHAN : 5
TERIMA KASIH
Press any key to continue
```

Gambar 2.5 Output Program Ketika Memilih Quit

C. Pembahasan

1. Baris [1]

Berfungsi sebagai header file untuk mengimpor library input/output standar.

2. Baris [2]

Mengimpor library untuk fungsi getch() yang digunakan untuk menahan layar.

3. Baris [3]

Mengimpor library untuk fungsi system("cls") yang digunakan untuk membersihkan layar konsol.

4. Baris [5]

Mendefinisikan konstanta MAX dengan nilai 20, yang digunakan sebagai kapasitas maksimal elemen yang dapat ditampung oleh Stack.

5. Baris [7]

Agar tidak perlu menulis std::cout, std::string, cukup cout, string saja.

6. Baris [9] sampai [13]

Mendeklarasikan struct Stack dengan dua field, yaitu Atas bertipe int yang berfungsi sebagai penunjuk indeks elemen teratas stack, dan Data[MAX] yang merupakan array integer berkapasitas 20 elemen sebagai tempat penyimpanan data stack.

7. Baris [14]

Mendeklarasikan variabel global Tumpuk bertipe Stack. Karena bersifat global, variabel ini dapat diakses oleh seluruh fungsi dalam program.

8. Baris [17] sampai [27]

Fungsi Kosong() berfungsi untuk memeriksa apakah stack dalam keadaan kosong. Apabila nilai Tumpuk.Atas sama dengan -1, fungsi mengembalikan nilai 1 (True) yang menandakan stack kosong. Sebaliknya, fungsi mengembalikan nilai 0 (False).

9. Baris [29] sampai [39]

Fungsi Penuh() berfungsi untuk memeriksa apakah stack sudah penuh. Apabila nilai Tumpuk.Atas sama dengan MAX-1 (yaitu 19), fungsi

mengembalikan nilai 1 (True) yang menandakan stack penuh. Sebaliknya, fungsi mengembalikan nilai 0 (False).

10. Baris [41] sampai [59]

Fungsi Push() berfungsi untuk memasukkan data baru ke dalam stack. Program memeriksa kondisi stack terlebih dahulu, apabila stack kosong atau belum penuh, nilai Tumpuk.Atas dinaikkan satu terlebih dahulu, kemudian data disimpan pada indeks Tumpuk.Atas yang baru. Apabila stack sudah penuh, program mencetak pesan "Stack telah Penuh!!!".

11. Baris [61] sampai [72]

Fungsi Pop() berfungsi untuk mengambil dan menghapus elemen teratas stack. Apabila stack tidak kosong, program mencetak data pada indeks Tumpuk.Atas, kemudian nilai Tumpuk.Atas dikurangi satu sebagai tanda bahwa elemen tersebut telah dihapus. Apabila stack kosong, program mencetak pesan "Stack Kosong!!!".

12. Baris [74] sampai [88]

Fungsi Cetak_Stack() berfungsi untuk menampilkan seluruh isi stack. Perulangan for dimulai dari indeks Tumpuk.Atas hingga indeks 0, sehingga data ditampilkan dari elemen teratas ke elemen terbawah. Apabila stack kosong, program mencetak pesan "Stack Kosong!!!".

13. Baris [90] sampai [94]

Fungsi Bersihkan_Stack() berfungsi untuk mereset stack dengan cara mengubah nilai Tumpuk.Atas kembali ke -1, sehingga stack dianggap kosong tanpa perlu menghapus data secara fisik satu per satu.

14. Baris [96] sampai [99]

Fungsi Inisialisasi() berfungsi untuk menyiapkan stack sebelum digunakan pertama kali dengan mengatur nilai Tumpuk.Atas ke -1, yang menandakan kondisi stack awal dalam keadaan kosong.

15. Baris [101] sampai [103]

Fungsi main() merupakan fungsi utama program, tempat program mulai dijalankan. Baris pertama langsung memanggil fungsi Inisialisasi() untuk mengatur nilai Tumpuk.Atas ke 1 sebelum stack digunakan.

16. Baris [104] sampai [105]

Mendeklarasikan dua variabel lokal bertipe integer, yaitu Pilihan untuk menyimpan pilihan menu dari pengguna, dan Data untuk menyimpan nilai yang akan dipush ke stack. Perulangan do-while kemudian dimulai, yang berarti blok di dalamnya akan dieksekusi minimal satu kali sebelum kondisinya diperiksa.

17. Baris [107] sampai [115]

Menampilkan menu utama program ke layar yang terdiri dari 5 pilihan operasi stack, kemudian meminta pengguna memasukkan pilihannya yang disimpan ke variabel Pilihan.

18. Baris [118] sampai [122]

Apabila pengguna memilih 1, program meminta pengguna memasukkan nilai integer yang disimpan ke variabel Data, kemudian memanggil fungsi Push(Data) untuk memasukkan nilai tersebut ke dalam stack.

19. Baris [124] sampai [126]

Apabila pengguna memilih 2, program memanggil fungsi Pop() untuk mengambil dan menghapus elemen teratas dari stack.

20. Baris [128] sampai [130]

Apabila pengguna memilih 3, program memanggil fungsi Cetak_Stack() untuk menampilkan seluruh isi stack dari elemen teratas hingga terbawah.

21. Baris [132] sampai [134]

Apabila pengguna memilih 4, program memanggil fungsi Bersihkan_Stack() untuk mereset stack dengan mengubah nilai Tumpuk.Atas kembali ke -1.

22. Baris [136] sampai [138]

Apabila pengguna memasukkan nilai selain 1 sampai 4, blok default dijalankan dan program mencetak pesan "TERIMA KASIH" sebagai tanda program akan segera keluar.

23. Baris [140] sampai [142]

Setelah setiap operasi selesai, program mencetak teks "Press any key to continue", kemudian getch() menahan layar hingga pengguna menekan sembarang tombol. Setelah tombol ditekan, system("cls") membersihkan

tampilan konsol sehingga menu ditampilkan kembali dalam keadaan bersih pada iterasi berikutnya.

24. Baris [143] sampai [145]

Kondisi `while (Pilihan < 5)` memeriksa apakah perulangan perlu dilanjutkan. Selama pengguna tidak memilih 5, menu akan terus ditampilkan. Apabila pengguna memilih 5, perulangan berhenti dan `return 0` mengembalikan nilai 0 ke sistem operasi, yang menandakan bahwa program telah selesai dijalankan dengan sukses tanpa kendala.

SOAL 3

Cobalah contoh program berikut, running dan analisis hasilnya!

A. Source Code

Tabel 3. Source Code Soal Nomor 3

1	#include <iostream>
2	#include <conio.h>
3	#include <stdlib.h>
4	
5	#define MAX 20
6	
7	using namespace std;
8	
9	struct Queue
10	{
11	int Front, Rear, Size;
12	char Q[MAX];
13	};
14	
15	Queue Antrian;
16	
17	int Kosong()
18	{
19	if (Antrian.Front == Antrian.Rear)
20	{
21	return 1;
22	}
23	else
24	{
25	return 0;
26	}
27	}
28	
29	int Penuh()
30	{
31	if ((Antrian.Rear + 1) % Antrian.Size ==
32	Antrian.Front)
33	{
34	return 1;
35	} else{
36	return 0;
37	}
38	}

```

39
40 void INSERT(char huruf)
41 {
42     if (Penuh() == 1)
43     {
44         cout << "Queue Penuh!!!" << endl;
45     }
46     else
47     {
48         Antrian.Q[Antrian.Rear] = huruf;
49         cout << "Data: " <<
Antrian.Q[Antrian.Rear] << " masuk ke dalam
Queue" << endl;
50         Antrian.Rear = (Antrian.Rear + 1) %
Antrian.Size;
51     }
52 }
53
54 void DELETE()
55 {
56     if (Kosong() == 1)
57     {
58         cout << "Queue kosong!!!" << endl;
59     }
60     else
61     {
62         cout << "Data yang dihapus: " <<
Antrian.Q[Antrian.Front] << endl;
63         Antrian.Front = (Antrian.Front + 1) %
Antrian.Size;
64     }
65 }
66
67 void CETAKLAYAR()
68 {
69     if (Kosong() == 1)
70     {
71         cout << "Queue kosong!!!" << endl;
72     }
73     else
74     {
75         int i = Antrian.Front;
76         while(i != Antrian.Rear)
77         {
78             cout << "Queue ke-" << i << " = " <<

```

```

Antrian.Q[i] << endl;
79         i = (i + 1) % Antrian.Size;
80     }
81 }
82 }
83
84 void RESET()
85 {
86     Antrian.Front = 0;
87     Antrian.Rear = 0;
88     Antrian.Size = MAX;
89     cout << "Queue telah di-reset" << endl;
90 }
91
92 void Inisialisasi()
93 {
94     Antrian.Front = 0;
95     Antrian.Rear = 0;
96     Antrian.Size = MAX;
97 }
98
99 int main()
100 {
101     Inisialisasi();
102     int Pilihan;
103     char huruf;
104     do{
105         cout << "\nQUEUE" << endl;
106         cout << "======" << endl;
107         cout << "1. INSERT" << endl;
108         cout << "2. DELETE" << endl;
109         cout << "3. CETAK QUEUE" << endl;
110         cout << "4. RESET QUEUE" << endl;
111         cout << "5. QUIT" << endl;
112         cout << "PILIHAN : "; cin >> Pilihan;
113         switch (Pilihan)
114         {
115             case 1:
116                 cout << "Masukkan Nilai: "; cin >>
huruf;
117                 INSERT(huruf);
118                 break;
119
120             case 2:
121                 DELETE();

```

```

122         break;
123
124     case 3:
125         CETAKLAYAR();
126         break;
127
128     case 4:
129         RESET();
130         break;
131
132     default:
133         cout << "TERIMA KASIH" << endl;
134         break;
135     }
136     cout << "Press any key to continue";
137     getch();
138     system("cls");
139 }
140 while (Pilihan < 5);
141 return 0;
142 }

```

B. Output Program

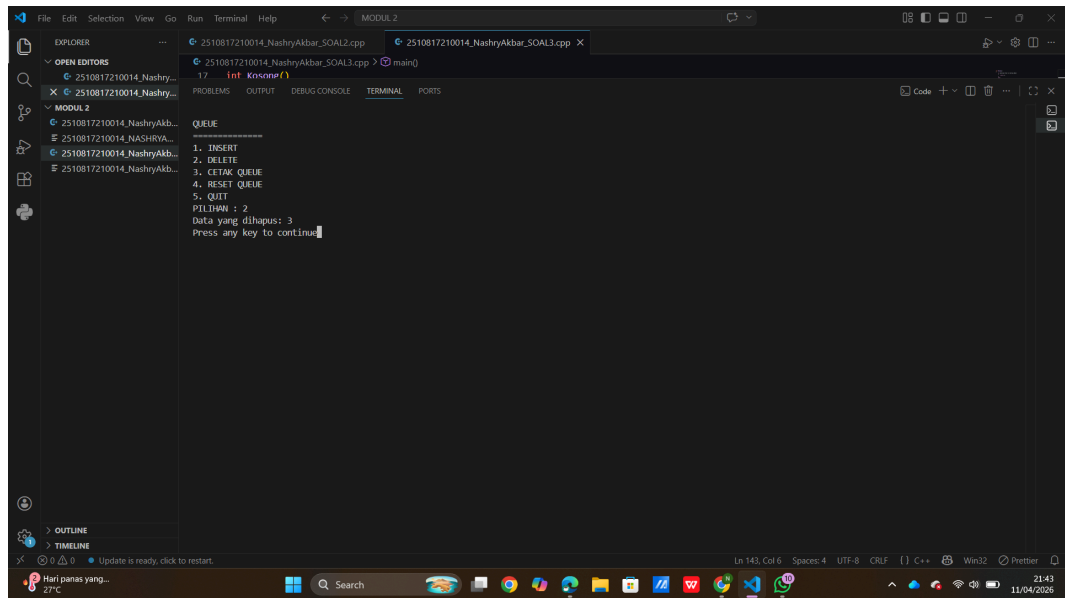
```

PS C:\Users\USER\OneDrive\Documents\Git\Praktikum-Algorithm\Modul 2> cd "C:\Users\USER\OneDrive\Documents\Git\Praktikum-Algorithm\Modul 2"; if ($?) { g++ 2510817210014_nashryakbar_SOAL3.cpp -o 2510817210014_nashryakbar_SOAL3 }; if ($?) { .\2510817210014_nashryakbar_SOAL3 }

QUEUE
=====
1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET QUEUE
5. QUIT
PILIHAN : 1
Masukkan Nilai: 3
Data: 3 masuk ke dalam Queue
Press any key to continue

```

Gambar 3.1 Output Program Ketika Memilih Insert



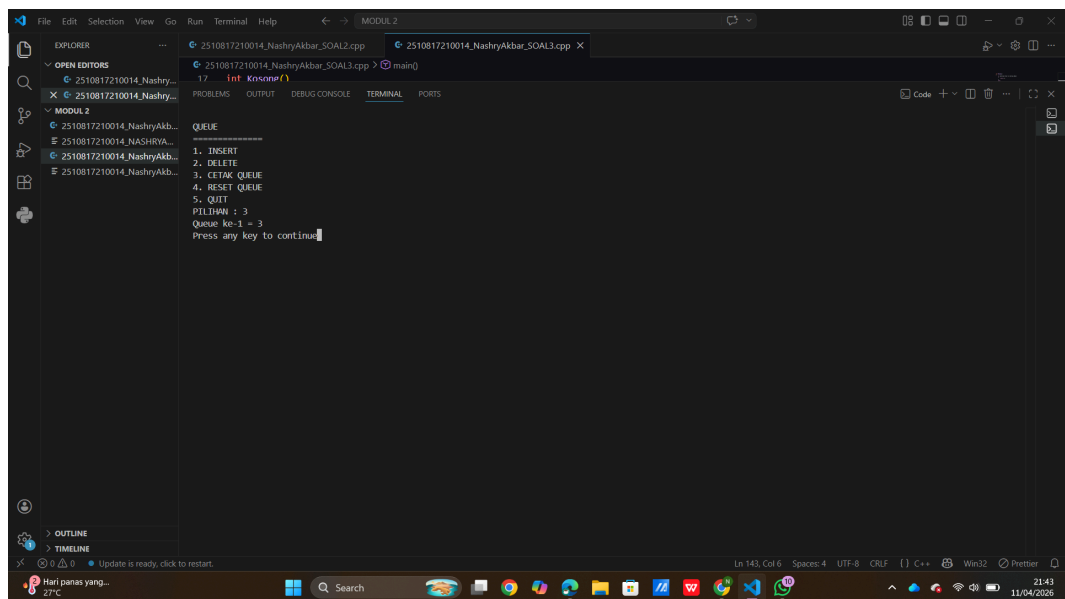
```
File Edit Selection View Go Run Terminal Help
2510817210014_NashryAkbar_SOAL2.cpp 2510817210014_NashryAkbar_SOAL3.cpp X
OPEN EDITORS
2510817210014_Nashry...
X 2510817210014_Nashry...
MODUL 2
2510817210014_NashryAk...
2510817210014_NASHRYA...
2510817210014_NashryAk...
2510817210014_NashryAk...
OUTLINE
TIMELINE
Update is ready, click to restart.
Ln 143, Col 6, Spaces: 4, UTF-8, CRLF, C++, Win32, Prettier
Hari panas yang... 27°C
21:43 11/04/2026
```

17 `int Kosoner()`

QUEUE

1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET QUEUE
5. QUIT
PILIHAN : 2
Data yang dihapus: 3
Press any key to continue

Gambar 3.2 Output Program Ketika Memilih Delete



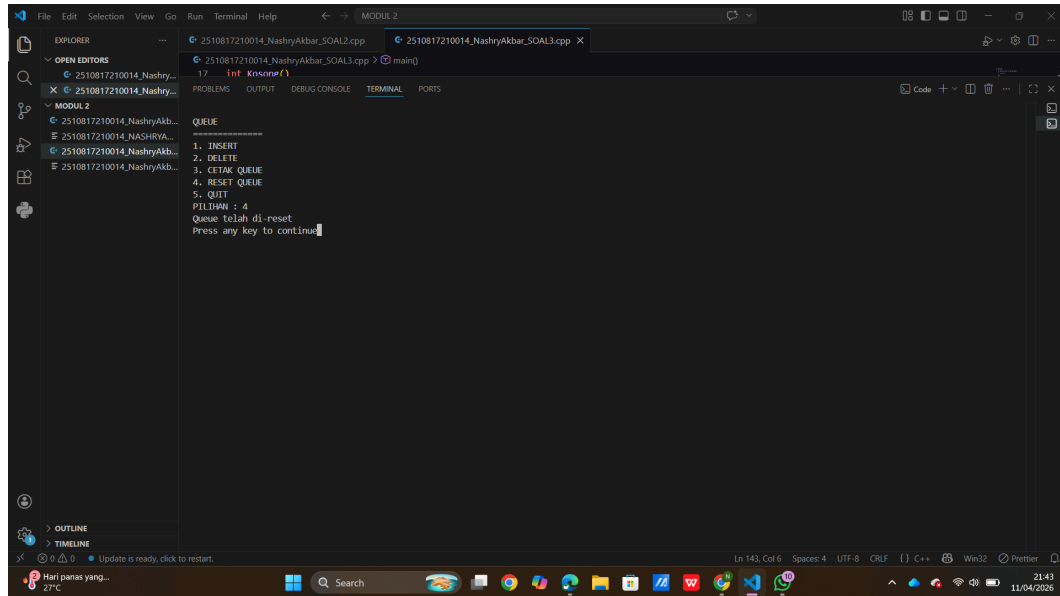
```
File Edit Selection View Go Run Terminal Help
2510817210014_NashryAkbar_SOAL2.cpp 2510817210014_NashryAkbar_SOAL3.cpp X
OPEN EDITORS
2510817210014_Nashry...
X 2510817210014_Nashry...
MODUL 2
2510817210014_NashryAk...
2510817210014_NASHRYA...
2510817210014_NashryAk...
2510817210014_NashryAk...
OUTLINE
TIMELINE
Update is ready, click to restart.
Ln 143, Col 6, Spaces: 4, UTF-8, CRLF, C++, Win32, Prettier
Hari panas yang... 27°C
21:43 11/04/2026
```

17 `int Kosoner()`

QUEUE

1. INSERT
2. DELETE
3. CETAK QUEUE
4. RESET QUEUE
5. QUIT
PILIHAN : 3
Queue ke-1 = 3
Press any key to continue

Gambar 3.3 Output Program Ketika Memilih Cetak Queue



```
File Edit Selection View Go Run Terminal Help
2510817210014_NashryAkbar_SOAL2.cpp 2510817210014_NashryAkbar_SOAL3.cpp X
OPEN EDITORS
2510817210014_Nashry...
X 2510817210014_Nashry...
MODUL 2
2510817210014_NashryAk...
2510817210014_NASHRYA...
2510817210014_NashryAk...
2510817210014_NashryAk...
OUTLINE
TIMELINE
Update is ready, click to restart.
Ln 143, Col 6, Spaces: 4, UTF-8, CRLF, C++, Win32, Prettier
Hari panas yang... 27°C
21:43 11/04/2028
```

17 `int Kosong()`

QUEUE

1. INSERT

2. DELETE

3. CETAK QUEUE

4. RESET QUEUE

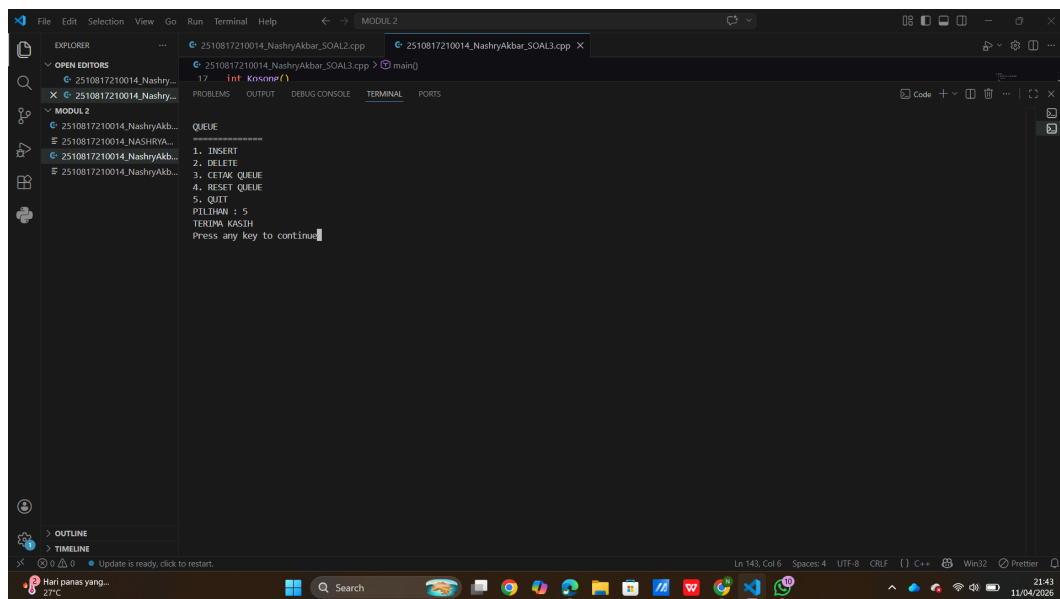
5. QUIT

PILIHAN : 4

Queue telah di-reset

Press any key to continue

Gambar 3,4 Output Program Ketika Memilih Reset Queue



```
File Edit Selection View Go Run Terminal Help
2510817210014_NashryAkbar_SOAL2.cpp 2510817210014_NashryAkbar_SOAL3.cpp X
OPEN EDITORS
2510817210014_Nashry...
X 2510817210014_Nashry...
MODUL 2
2510817210014_NashryAk...
2510817210014_NASHRYA...
2510817210014_NashryAk...
2510817210014_NashryAk...
OUTLINE
TIMELINE
Update is ready, click to restart.
Ln 143, Col 6, Spaces: 4, UTF-8, CRLF, C++, Win32, Prettier
Hari panas yang... 27°C
21:43 11/04/2028
```

17 `int Kosong()`

QUEUE

1. INSERT

2. DELETE

3. CETAK QUEUE

4. RESET QUEUE

5. QUIT

PILIHAN : 5

TERIMA KASIH

Press any key to continue

Gambar 3.5 Output Program Ketika Memilih Quit

C. Pembahasan

1. Baris [1]

Berfungsi sebagai header file untuk mengimpor library input/output standar.

2. Baris [2]

Mengimpor library untuk fungsi `getch()` yang digunakan untuk menahan layar.

3. Baris [3]
Mengimpor library untuk fungsi `system("cls")` yang digunakan untuk membersihkan layar konsol.
4. Baris [5]
Mendefinisikan konstanta MAX dengan nilai 20, yang digunakan sebagai kapasitas maksimal elemen yang dapat ditampung oleh Queue.
5. Baris [7]
Agar tidak perlu menulis `std::cout`, `std::string`, cukup `cout`, `string` saja.
6. Baris [9] sampai [13]
Mendeklarasikan struct Queue dengan empat field. Front bertipe int sebagai penunjuk indeks elemen terdepan antrian, Rear bertipe int sebagai penunjuk indeks posisi elemen terbelakang antrian, Size bertipe int untuk menyimpan ukuran maksimal queue, dan Q[MAX] merupakan array char berkapasitas 20 elemen sebagai tempat penyimpanan data antrian.
7. Baris [15]
Mendeklarasikan variabel global Antrian bertipe Queue. Karena bersifat global, variabel ini dapat diakses oleh seluruh fungsi dalam program.
8. Baris [17] sampai [27]
Fungsi Kosong() berfungsi untuk memeriksa apakah queue dalam keadaan kosong. Apabila nilai Antrian.Front sama dengan Antrian.Rear, fungsi mengembalikan nilai 1 (True) yang menandakan queue kosong. Sebaliknya, fungsi mengembalikan nilai 0 (False).
9. Baris [29] sampai [39]
Fungsi Penuh() berfungsi untuk memeriksa apakah queue sudah penuh. Kondisi $(\text{Antrian.Rear} + 1) \% \text{Antrian.Size} == \text{Antrian.Front}$ merupakan teknik circular buffer, yaitu apabila posisi setelah Rear sudah menyentuh posisi Front, maka queue dinyatakan penuh dan fungsi mengembalikan nilai 1 (True). Sebaliknya, fungsi mengembalikan nilai 0 (False).
10. Baris [41] sampai [53]
Fungsi INSERT() berfungsi untuk memasukkan data baru ke dalam queue. Apabila queue penuh, program mencetak pesan "Queue Penuh!!!". Apabila belum penuh, data huruf disimpan pada indeks Antrian.Rear,

kemudian nilai `Antrian.Rear` digeser ke posisi berikutnya menggunakan operasi modulo `% Antrian.Size` agar pergerakan indeks bersifat melingkar (circular).

11. Baris [55] sampai [66]

Fungsi `DELETE()` berfungsi untuk mengambil dan menghapus elemen terdepan queue. Apabila queue kosong, program mencetak pesan "Queue kosong!!!". Apabila tidak kosong, program mencetak data pada indeks `Antrian.Front`, kemudian nilai `Antrian.Front` digeser ke posisi berikutnya menggunakan operasi modulo agar pergerakan indeks tetap bersifat melingkar.

12. Baris [68] sampai [83]

Fungsi `CETAKLAYAR()` berfungsi untuk menampilkan seluruh isi queue. Variabel `i` dimulai dari posisi `Front`, kemudian perulangan `while` berjalan selama `i` belum mencapai `Rear`. Setiap iterasi mencetak data pada indeks `i`, lalu `i` digeser ke posisi berikutnya menggunakan operasi modulo. Apabila queue kosong, program mencetak pesan "Queue kosong!!!".

13. Baris [85] sampai [91]

Fungsi `RESET()` berfungsi untuk mereset queue ke kondisi awal dengan cara mengatur kembali nilai `Front` dan `Rear` ke 0, serta `Size` ke nilai `MAX`. Setelah reset, queue dianggap kosong karena `Front` dan `Rear` kembali berada di posisi yang sama.

14. Baris [93] sampai [98]

Fungsi `Inisialisasi()` berfungsi untuk menyiapkan queue sebelum digunakan pertama kali dengan mengatur nilai `Front` dan `Rear` ke 0 serta `Size` ke `MAX`, yang menandakan kondisi queue awal dalam keadaan kosong.

15. Baris [100] sampai [102]

Fungsi `main()` merupakan fungsi utama program, tempat program mulai dijalankan. Baris pertama langsung memanggil fungsi `Inisialisasi()` untuk menyiapkan queue sebelum digunakan.

16. Baris [103] sampai [104]

Mendeklarasikan dua variabel lokal, yaitu Pilihan bertipe int untuk menyimpan pilihan menu dari pengguna, dan huruf bertipe char untuk menyimpan karakter yang akan di-insert ke dalam queue.

17. Baris [105] sampai [115]

Perulangan do-while dimulai, yang berarti blok di dalamnya akan dieksekusi minimal satu kali. Program menampilkan menu utama yang terdiri dari 5 pilihan operasi queue, kemudian meminta pengguna memasukkan pilihannya yang disimpan ke variabel Pilihan.

18. Baris [118] sampai [122]

Apabila pengguna memilih 1, program meminta pengguna memasukkan sebuah karakter yang disimpan ke variabel huruf, kemudian memanggil fungsi INSERT(huruf) untuk memasukkan karakter tersebut ke dalam queue.

19. Baris [124] sampai [126]

Apabila pengguna memilih 2, program memanggil fungsi DELETE() untuk mengambil dan menghapus elemen terdepan dari queue.

20. Baris [128] sampai [130]

Apabila pengguna memilih 3, program memanggil fungsi CETAKLAYAR() untuk menampilkan seluruh isi queue dari posisi Front hingga sebelum posisi Rear.

21. Baris [132] sampai [134]

Apabila pengguna memilih 4, program memanggil fungsi RESET() untuk mereset queue ke kondisi awal dengan mengatur kembali nilai Front, Rear, dan Size.

22. Baris [136] sampai [138]

Apabila pengguna memasukkan nilai selain 1 sampai 4, blok default dijalankan dan program mencetak pesan "TERIMA KASIH" sebagai tanda program akan segera keluar.

23. Baris [140] sampai [142]

Setelah setiap operasi selesai, program mencetak teks "Press any key to continue", kemudian getch() menahan layar hingga pengguna menekan sembarang tombol. Setelah tombol ditekan, system("cls") membersihkan

tampilan konsol sehingga menu ditampilkan kembali dalam keadaan bersih pada iterasi berikutnya.

24. Baris [143] sampai [144]

Kondisi `while (Pilihan < 5)` memeriksa apakah perulangan perlu dilanjutkan. Selama pengguna tidak memilih 5, menu akan terus ditampilkan. Apabila pengguna memilih 5, perulangan berhenti dan `return 0` mengembalikan nilai 0 ke sistem operasi, yang menandakan bahwa program telah selesai dijalankan dengan sukses tanpa kendala.

LINK GITHUB

<https://github.com/Nashry4/Praktikum-Algoritma>